Learn to Design and Verify a 6-stage pipelined RISC-V CPU

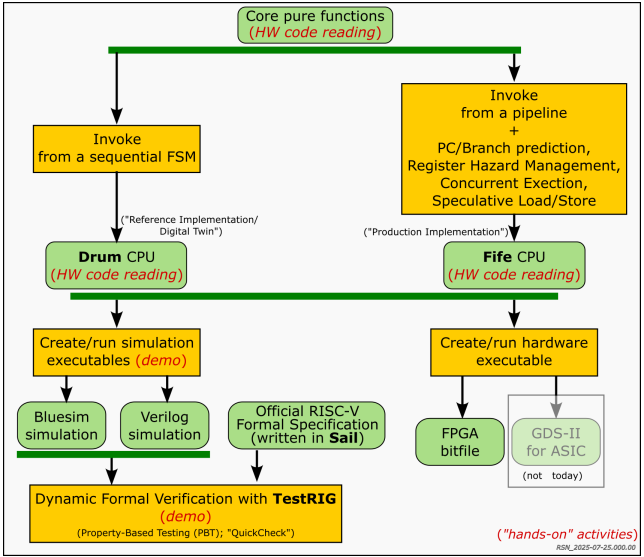
Rishiyur S. Nikhil, Cofounder and CTO, Bluespec, Inc.

October 22, 2025

Tutorial at Developer's Day, RISC-V Summit, Santa Clara CA

Slides version: 1.00

The Plan for Today



"Hands on":

- We will read the code together
- We create and run simulation executables together
- We will verify the design together

(If you haven't installed the tools yet, please just listen and follow; hopefully you will try everything on your own, later)

Not "Hands on" (lecture only):

- How to build and run on FPGA

"But I'm not familiar with hardware design" or
"I'm not familiar with the language used" ...
... don't worry:

- For "Core pure functions" and **Drum**, the code looks just like a conventional programming language
- **Fife** will introduce pipelining and pipeline techniques; we will explain the code in more detail.

Prerequisites

We *do* assume that you are familiar with:

- Some ISA (Instruction Set Architecture): Machine Instructions and/or assembly-language (RISC-V or other).

  We're not going to explain what is a general-purpose register, what is a Program Counter, what is an arithmetic instruction, what is a BRANCH or JAL/JALR instruction, what is a LOAD/STORE instruction, etc.; we're assuming you already know these things.

- General coding (in C, Python, Go, Rust, Haskell, Verilog, SystemVerilog, ... whatever), so you can quickly grasp new, unfamiliar code.
  Familiarity with enum and struct types, functions, if-then-else.
  General knowledge of object-oriented programming is useful.

We *do not* assume that you are familiar with:

- Hardware design, or any hardware design language (Verilog, SystemVerilog, Bluespec BSV or BH, Chisel, ...)
- Pipelined micro-architecture of CPUs

On the Tutorial Web Page (`https://github.com/rsnikhil/Tutorial_RISCV_Summit_2025`), we've asked you to install certain tools so that you can examine code and build and run things along with me during this tutorial.

If you haven't done this already, then please just sit back and follow the lecture and demos, for now. We hope that you will be motivated to later install the tools and try everything out for real.

Resources for this tutorial (all free and open-source)

This Tutorial:
Repo: **https://github.com/rsnikhil/Tutorial_RISCV_Summit_2025**
This slide deck: Slides_Nikhil_RISCV_Summit_Tutorial.pdf
Tool installation guide: Doc/Resources.html

Please git-clone this for your local copy of the slides

Textbook and teaching resources
Suitable for medium/advanced Undergraduate or early Graduate students
Repo: **https://github.com/rsnikhil/Learn_Bluespec_and_RISCV_Design**
Textbook (350+ pages): Book_BLang_RISCV.pdf
Includes many exercises.
All topics in this tutorial (and much more) covered in detail.
Full source code: for **Fife** and **Drum** CPUs.
Makefiles etc. to build Bluesim and Verilog simulations
Lecture slide PDFs (per book chapter)
Code solutions for exercises

Please git-clone this for your local copy of the CPU source codes

Required (Bluesim simulation, Verilog generation)

Tool: *bsc* compiler (BSV to Bluesim/Verilog)
https://github.com/B-Lang-org/bsc

Optional

Tool: *Verilator* (for Verilog simulation)
https://github.com/verilator/verilator

Tool: TestRIG for verifying RISC-V CPUs
From University of Cambridge
Uses (requires) **Haskell QuickCheck**
https://github.com/CTSRD-CHERI/TestRIG

Tool: Official RISC-V Formal Specification
Written in **Sail**.
Compiled with **OCaml** into a simulator
https://github.com/riscv/sail-riscv

RSN_2025-10-16.000.00

We have a **lot** *to get through,*
And just two hours to blow
It'll be a bit of a fire hose
Only a launchpad for more.

So that's the plan;
Don't panic, stay steady!
It's really not so hard,
Let's go! Are you ready!

I'm sure you'll have more questions or follow-ups;
please catch me at this Summit through tomorrow,
or email me later:

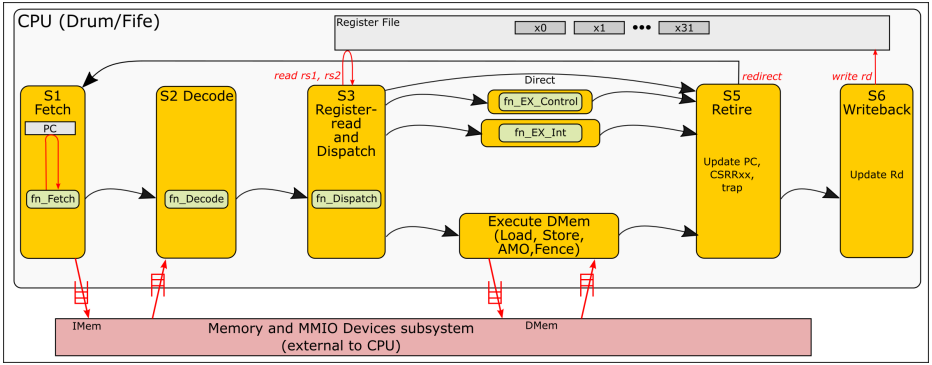
`rsnikhil@pm.me`
`nikhil@bluespec.com`

Contents

Core pure functions

Across many an implementation
Certain functions are the same
So, "Write once, Use many";
This'll be our game.

Basic steps in executing an instruction (any CPU, incl. **Drum** and **Fife**)



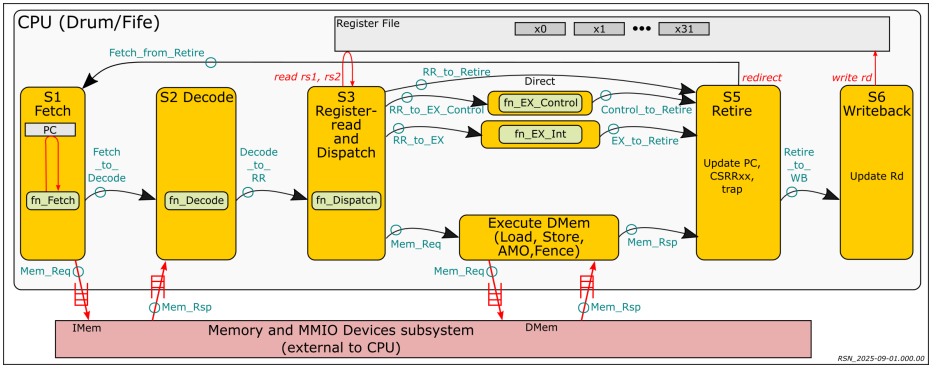
This diagram illustrates the core steps in executing any instruction. The "`fn_XXX`" boxes indicate core functions within each step.

The "Memory and MMIO Devices subsystem" is *external* to the CPU and is not our focus today (we will in fact use C code to "model" this in simulation). The CPU sends requests to memory; for each request and the memory sends a response back to the CPU, *after an unknown latency*. This is an "asynchronous split-phase transaction."

Memory-access latency can vary dynamically due to possible caching, congestion, varying-length paths to different memory units/devices *etc.*.

The CPU correctness should be robust to this varying latency.

Core pure functions (shared by **Drum** and **Fife**)



This diagram annotates each inter-step/stage arrow with the "struct" type it communicates (in green text).

The core functions are pure functions, and are shared by **Drum** and **Fife**:

E.g.,

`fn_Fetch`: (pure) function from the PC to a pair of structs of type `Mem_Req` and `Fetch_to_Decode`
`fn_Decode`: (pure) function from a pair of structs of type `Fetch_to_Decode` and `Mem_Rsp` to a struct of type `Decode_to_RR`

... and so on.

Let's read some code together: Memory request/response types

Repository:	`https://github.com/rsnikhil/Learn_Bluespec_and_RISCV_Design`
File:	`Code/src_Common/Instr_Bits.bsv`
Focus on:	`funct5_{LOAD/STORE}, instr_funct5()`
	`funct3_{LB,LH,LW,SB,SH,SW}, instr_funct3()`

- These are the bit-encodings of fields in RISC-V LOAD and STORE instructions that indicate whether the instruction is a LOAD or a STORE, and whether it is for a Byte (B), Halfword (H), Word (W) or doubleword (D).

  These are defined in *The RISC-V Instruction Set Manual Volume I, Unprivileged Architecture*, at `https://drive.google.com/file/d/1uviu1nH-tScFfgrovvFCrj7Omv8tFtkp/view` *Chapter 35: RV32/64G Instruction Set Listings*

Other `funct5_*` definitions are for AMO instructions, distinguishing between FETCH and LOAD, *etc.*

Let's read some code together: struct Mem_Req (memory requests)

Repository:	https://github.com/rsnikhil/Learn_Bluespec_and_RISCV_Design
File:	Code/src_Common/Mem_Req_Rsp.bsv
Focus on:	struct Mem_Req

- In struct Mem_Req, the req_type field has type Mem_Req_Type, which is defined earlier in the file to be a synonym for Bit #(5), *i.e.,* a 5-bit value, corresponding to the funct5_* definitions from the previous slide (LOAD? STORE?).
- The req_size field has type Mem_Req_Size which is defined earlier in the file as an "enum' type indicating whether a memory request is for 1, 2, 4 or 8 bytes.
- The addr and data fields are of type Bit #(64), *i.e.,* 64-bit wide values.
  The data field is only relevant when the request is for a STORE instruction; it can be ignored for Fetches, LOADs, and LRs.

Note:

We could have used Bit #(XLEN) for these types, where XLEN is defined in file Code/src_Common/Arch.bsv to be either 32 (for RISC-V RV32 implementation) or 64 (for RISC-V RV64 implementation). Decoupling the CPU's XLEN from the memory port width is useful for many reasons:

- A wider memory fetch port allows pre-fetching, and facilitates compressed-instruction parsing.
- A wider memory load/store port may be needed for double-precision floating point in an RV32 CPU.

- The remaining fields of struct Mem_Req are not relevant for the moment. The epoch field is only relevant for **Fife**, and the remaining fields are only for debugging.

Let's read some code together: struct Mem_Rsp (memory responses)

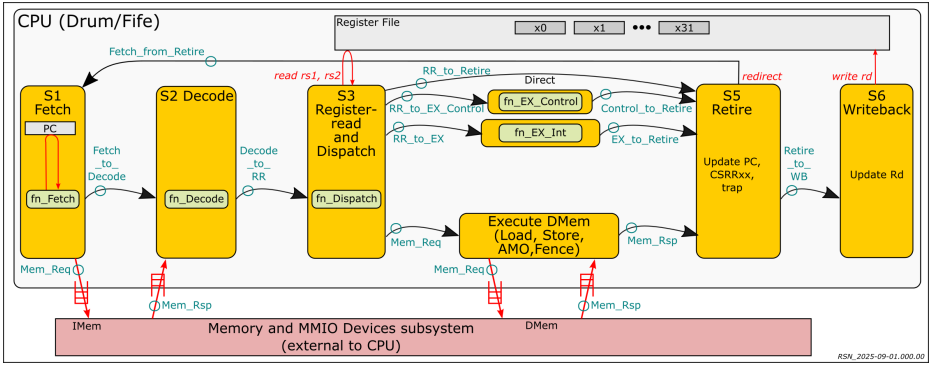
Repository:	https://github.com/rsnikhil/Learn_Bluespec_and_RISCV_Design
File:	Code/src_Common/Mem_Req_Rsp.bsv
Focus on:	struct Mem_Rsp

- The field rsp_type has type Mem_Rsp_Type which is defined earlier in the file to be and enum with values MEM_RSP_OK, MEM_RSP_MISALIGNED, MEM_RSP_ERR, or MEM_REQ_DEFERRED.

  A memory request may succeed, or it may fail because it was directed at an unsupported address ("wild" access), or because it was misaligned and the external memory system does not support misaligned addresses in requests, or some other reason. The first three response types handle these situations.

  MEM_REQ_DEFERRED can be ignored for now; it is only relevant in Fife.
- The field `data` is a 64-bit field containing data in the case of a memory access that returns data from memory (FETCH, LOAD, AMO).
- The remaining fields are only for debugging.

(Repeating earlier diagram, for convenience)



Let's read some code together: struct `Fetch_to_Decode`

Repository:	`https://github.com/rsnikhil/Learn_Bluespec_and_RISCV_Design`
File:	`Code/src_Common/Inter_Stage.bsv`
Focus on:	struct `Fetch_to_Decode`

- The notation `Bit #(XLEN)` represents an `XLEN`-wide bit-vector. Parameterization by `XLEN` allows using the same code for both RV32 and RV64 implementations.
- In the `Fetch_to_Decode` struct, the only field of interest for the moment is `pc`, the value of the Program Counter. It is forwarded to later steps/stages because:
  - It is needed by AUIPC and Branch instructions to compute PC-relative values
  - It may be needed by JUMP instructions (JAL/JALR) to save a "return address"
  - In case of a trap or an interrupt, the PC needs to be saved in the MEPC CSR
- The remaining fields (`predicted_pc`, `epoch`, `inum`, `halt_sentinel`, ...) can be ignored for the moment—some are only needed in **Fife**, and some are only needed for debugger-control.

Let's read some code together: `fn_Fetch`

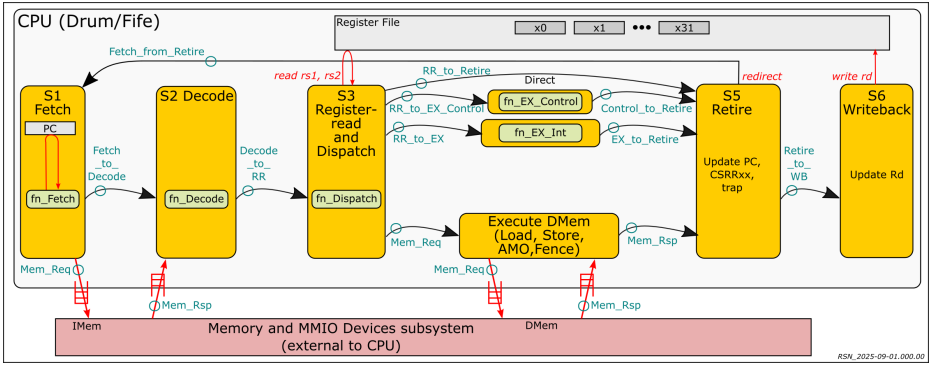
Repository:	`https://github.com/rsnikhil/Learn_Bluespec_and_RISCV_Design`
File:	`Code/src_Common/Fn_Fetch.bsv`
Focus on:	function `fn_Fetch`

- Conceptually: `fn_Fetch:` PC $\rightarrow$ (Memory-request, Info-to-Decode)
- Before the function definition is a definition of a struct `Result_F` representing the return-type of the function. It contains a pair of structs of type `Fetch_to_Decode` (to be sent to the Decode step/stage) and `Mem_Req` (to be sent to memory), respectively.

  Note: Unlike C functions, which can only return values of size $\leq$ 64 bits, **BSV** functions can return arbitrarily-wide values, including complete structs, vectors, nested structs and vectors, *etc.*.
- The return-type of the function `fn_Fetch` is `ActionValue #(Result_F)`. This specifies not only the type of the returned value (`Result_F`), but also that this function may have side-effects (`ActionValue #()`). This is useful both for user-documentation, but also for the compiler (side-effecting functions cannot be optimized as much as non-side-effecting ("pure") functions).
- Of the arguments, only `pc` is important, for now. `predicted_pc` and `epoch` are needed in **Fife**, and the remaining arguments are only for debugging.
- The function body is straightforward: it simply creates the `Fetch_to_Decode` and `Mem_Req` structs for the result. Note: the memory-request is a LOAD for a 4-byte value (a RISC-V instruction) at address `pc`.

  The `zeroExtend` on the address extends 32 bits to 64 bits.

(Repeating earlier diagram, for convenience)



Let's read some code together: fn_Decode

Repository:	https://github.com/rsnikhil/Learn_Bluespec_and_RISCV_Design
Files:	Code/src_Common/Inter_Stage.bsv
	Code/src_Common/Fn_Decode.bsv
Focus on:	struct Decode_to_RR, function fn_Decode

- Conceptually: fn_Decode: (Info-from-Fetch, Memory-response) → Info-to-Reg-Read (refer figure on Slide 8).
- This step/stage takes the incoming instruction and passes it along with some "classification" information. The classes are defined as values of the enum OpClass type (file Inter_Stage.bsv). Instructions of the SYSTEM and FENCE classes are sent directly to the Retire step/stage. CONTROL, INT and MEM-class instructions are sent into the corresponding fourth-stage execution paths shown in the figure on Slide 8.
- Struct Decode_to_RR (file Inter_Stage.bsv) represents the return-type of function fn_Decode. Some of the fields just pass through values that came in on the incoming Fetch_to_Decode struct.
- The has_rs1, has_rs2 and has_rd are booleans indicating whether the instructing reads one or two register values, and whether it writes a register value. The writes_mem field is true if the instruction writes to memory (STORE, SC, AMO instructions). The imm field is the "immediate" value from the instruction (different kinds of instructions encode immediate values differently).
- The exception field is true if the Fetch memory-response returned a memory error, or if the instruction itself is not legal. In this case, the cause and tval fields provide additional information needed for the trap that will be taken in the Retire step/stage.
- The fn_Decode function is just a big nested conditional that examines the instruction and fills in these fields appropriately. It uses help-functions like is_legal_LUI, is_legal_BRANCH, and so on, defined in Code/src_Common/Instr_Bits.bsv.

Interlude

Badger: Ok, ok, enough of this pseudo-code, I think I get it, but when are we going to start writing the actual hardware code?

Owl: This isn't pseudo-code; this *is* the actual hardware code!

Let's read some code together: The Register File (General-Purpose Registers, or GPRs) (1/3)

Repository:	`https://github.com/rsnikhil/Learn_Bluespec_and_RISCV_Design`
File:	`Code/src_Common/GPRs.bsv`
Focus on:	interface `GPRS_IFC`, module `mkGPRs`

- The RISC-V ISA requires 32 GPRs, where GPR 0 always reads as 0.
- **BSV** designs are composed of *modules*, each of which has an *interface*. These are similar to *objects* and *methods* in object-oriented programming languages.
- `GPRs_IFC` defines an interface with several methods.

  `read_rs1`: this method takes a 5-bit argument `rs1` which identifies a GPR, and returns an `xlen`-bit value which is the contents of that GPR. (And, similarly, `read_rs2`.)

  `write_rd`: this method takes a 5-bit argument `rs1` which identifies a GPR, and an `xlen`-bit argument that is a value, and writes that value into that GPR. The "return type" `Action` indicates that it has a side-effect, and that it does not return any value.

- `read_dm`, `write_dm`: these methods can be ignored for now; they are intended for an optional RISC-V Debug Module to access registers.

Let's read some code together: The Register File (General-Purpose Registers, or GPRs) (2/3)

Repository:	https://github.com/rsnikhil/Learn_Bluespec_and_RISCV_Design
File:	Code/src_Common/GPRs.bsv
Focus on:	interface GPRS_IFC, module mkGPRs

- mkGPRs defines a module that presents the GPRs_IFC interfaces.
- The first line instantiates the **BSV** library module mkRegFileFull; rf names the interface to this instance. This module has methods sub to read a register and .upd to update a register with new value.
- The next section initializes all the registers to 0 when coming out of reset. This is not strictly required by the RISC-V ISA, but for debugging it is useful to emerge from reset in a known state.

  We instantiate a standard **BSV** library register module mkReg that is initialized to zero, calling its interface rg_init_index.

  The rule rl_init is effectively a loop, counting up on rg_init_index; at iteration j it writes 0 to GPR[j] using the .upd method for register files. The rule condition ("(! initialized)") specifies when to terminate the loop.

  truncate() converts the 6-bit rg_init_index value to 5 bits needed for indexes into the register file. **BSV** has very strict type-checking; there are no silent truncations and extensions; without this truncate() we'd get a type error.
- The rest of the module specifies implementations of each method.

  "if (initialized)" is an *implicit condition* on each method, effectively blocking these methods from being invoked before complete initialization.

Let's read some code together: The Register File (General-Purpose Registers, or GPRs) (3/3)

Repository:	`https://github.com/rsnikhil/Learn_Bluespec_and_RISCV_Design`
File:	`Code/src_Common/GPRs.bsv`
Focus on:	interface `GPRS_IFC`, module `mkGPRs`

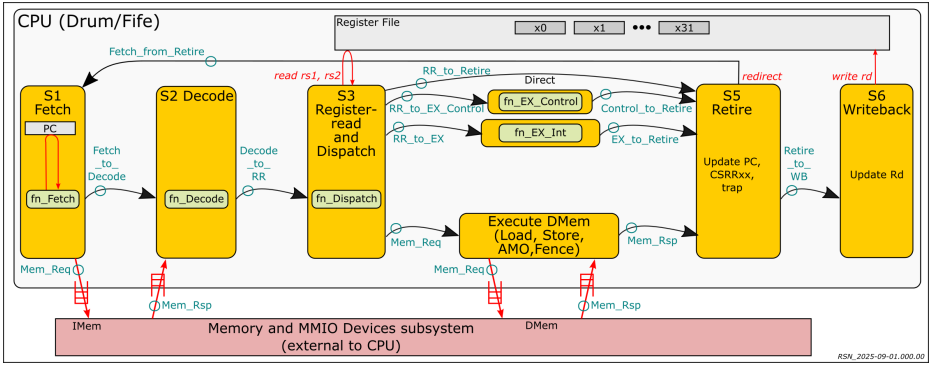
- The rest of the module specifies implementations of each method.
- "`if (initialized)`" is an *implicit condition* on each method, effectively blocking these methods from being invoked before complete initialization.
- Note that `write_rd` writes into the register file, but in `read_rs1/rs2`, if the index is zero, they ignore what's in the register file and return 0.

  Alternatively, `write_rd` write 0 if the index is zero, and `read_rs1/rs2` could blindly return the register value.

- The *bsc* compiler cannot generate Verilog RTL for module `mkGPRs` because it is *polymorphic*—it has interface buses of variable width (`xlen`).

  The module `mkGPRs_synth` fixes the variable `xlen` to be the constant `XLEN`. This module *can* be synthesized by *bsc*.

(Repeating earlier diagram, for convenience)

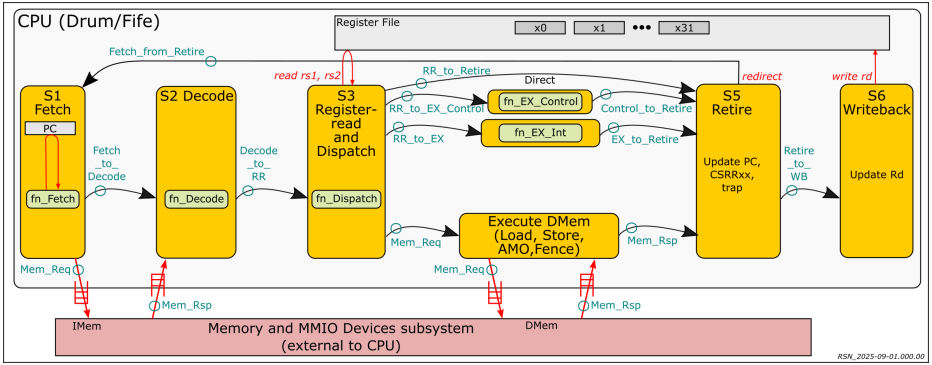


Let's read some code together: fn_Dispatch

Repository:	https://github.com/rsnikhil/Learn_Bluespec_and_RISCV_Design
Files:	Code/src_Common/Inter_Stage.bsv
	Code/src_Common/Fn_Dispatch.bsv
Focus on:	structs RR_to_Retire, RR_to_EX_Control, RR_to_EX
	function fn_Dispatch

- Conceptually: fn_Dispatch: Info-from-Decode → Info to (Retire, EX_Control, EX_Int, EX_DMem) (refer figure on Slide 8).
  The struct type Result_Dispatch contains four sub-structs for this purpose.
- Function fn_Dispatch() is straightforward, just constucting those four sub-structs and assembling them into the result struct. Each sub-struct is destined for one of the paths of execution shown in the figure.
- The RR_to_Retire struct is relevant for every instruction.
- The remaining three structs—RR_to_EX_Control, RR_to_EX, RR_to_EX_DMem—are each relevant only for a subset of instructions (Control, Integer and Memory, respectively), but this function fills them in blindly; those structs/fields will only be examined if relevant, so no harm is done if some of the values are bogus.

(Repeating earlier diagram, for convenience)

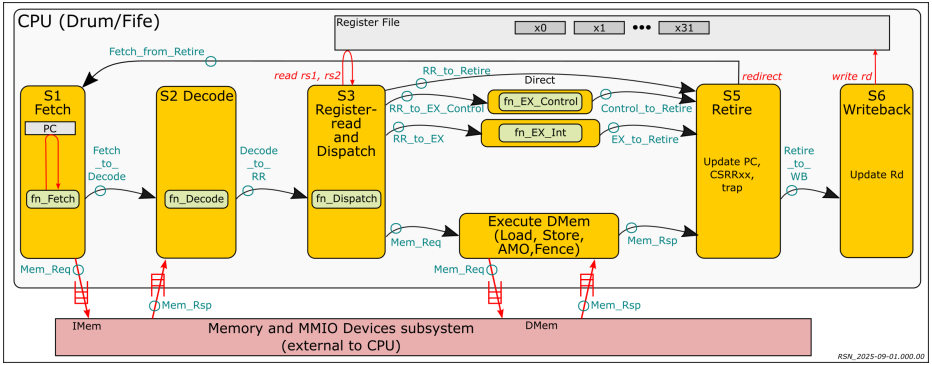


Let's read some code together: fn_EX_Control

Repository:	https://github.com/rsnikhil/Learn_Bluespec_and_RISCV_Design
Files:	Code/src_Common/Inter_Stage.bsv
	Code/src_Common/Fn_EX_Control.bsv
Focus on:	struct EX_Control_to_Retire
	function fn_EX_Control

- Conceptually: fn_EX_Control: Info-from-Dispatch → Info to Retire
  (refer figure on Slide 8).
- Function fn_Dispatch() is straightforward, implementing the semantics of RISC-V BRANCH (conditional branch), JAL and JALR instructions.
- It also checks for misaligned targets.

(Repeating earlier diagram, for convenience)



Let's read some code together: fn_EX_Int

Repository:	https://github.com/rsnikhil/Learn_Bluespec_and_RISCV_Design
Files:	Code/src_Common/Inter_Stage.bsv
	Code/src_Common/Fn_EX_Int.bsv
	Code/src_Common/IALU.bsv
Focus on:	struct EX_to_Retire
	function fn_EX_Int
	function fn_IALU

- Conceptually: fn_EX_Int: Info-from-Dispatch → Info to Retire
  (refer figure on Slide 8).
- Function fn_Int() is straightforward,
  implementing the semantics of RISC-V LUI, AUIPC and integer operations. The latter are packaged in function fn_IALU, which implements the semantics of RISC-V instructions:

(rs1, rs2)	ADD, SUB, SLL, SLT, SLTU, SRL, SRA, AND, OR, XOR
(rs1, immed)	ADDI, SLTI, SLTIU, ANDI, ORI, XORI, SLLI, SRLI, SRAI

Pause ...

Modulo minor syntactic differences, these types/functions look like types/functions written in many a popular software programming language.

The **BSV** code gets compiled by the *bsc* compiler into Verilog.

And they will be used, as-is, in both **Drum** and **Fife**.

Contents

Drum

To temporally sequence the functions
Is so easy, almost picayune;
Like writing an ordinary program,
A simple FSM; no danger of swoon.

Before girding up (later)
For the pipeline scrum,
Let's FSM our functions and,
et voila! We'll have Drum!

Drum: invoking the core pure functions from a sequential Finite State Machine (FSM)

This will give us a *hardware implementable* RISC-V CPU: **BSV** $\longrightarrow$ Verilog $\longrightarrow$ FPGA

- Not a very fast implementation, but still ≥ 1 *order of magnitude* faster than **BlueSim** or Verilog simulation (boot Linux in minutes instead of days).
- Can share the same system-surround (memory, devices) as Fife, and run the same software as Fife (because same `CPU_IFC` interface).
- More "obviously correct" than Fife: can be used as a verification reference (digital twin) for a more serious implementation such as Fife.
- More hospitable harness for debugging/verifying functional RISC-V parts (*i.e.*, the non-pipeline parts; the functions we've seen so far) than a pipelined CPU

Let's read some code together: exec_one_instr

Repository:	https://github.com/rsnikhil/Learn_Bluespec_and_RISCV_Design
Files:	Code/src_Drum/Drum_FSM.bsv
Focus on:	Stmt exec_one_instr

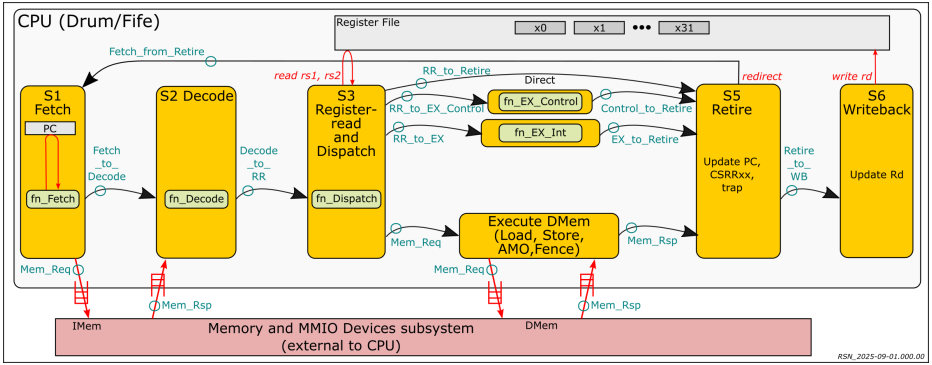
- "exec_one_instr" is a compound "statement" (of type Stmt) and is a straightforward reading of the steps needed to execute one instruction:
  Fetch → Decode → Register-read-and-dispatch → Execute-and-retire → Handle-exception-if-necessary
- Each step has been abstracted into an "action function" (such as a_Fetch) which we will study momentarily.
- The seq-endseq bracket indicate that its component actions should be performed sequentially (remember, **Drum** is not pipelined).
- The fourth action is itself an if-then-else corresponding to the four execution paths on Slide 8. Some of them use nested seq-endseq to sequence an execute action and a retire action.

Let's read some code together: Execute instructions sequentially, forever

Repository:	`https://github.com/rsnikhil/Learn_Bluespec_and_RISCV_Design`
Files:	`Code/src_Drum/Drum_FSM.bsv`
Focus on:	`mk_AutoFSM`

- "`mk_AutoFSM`" is a **BSV** library function that here embeds "`exec_one_instr`" into an infinite loop, thus fetching and executing instructions forever (the third arm of the if-then-else).
- Please ignore the first arm of the if-then-else: that is meant for debugger control.
- The second arm prioritizes taking an interrupt (if one is pending) over executing the next instruction. Thus, we always take interrupts *between* instructions.

(Repeating earlier diagram, for convenience)



Let's read some code together: the interface CPU_IFC for **Fife** and **Drum**

Repository:	https://github.com/rsnikhil/Learn_Bluespec_and_RISCV_Design
Files:	Code/src_Common/CPU_IFC.bsv
Focus on:	interface CPU_IFC

- The sub-interface fo_IMem_req, of type FIFOF_O #(Mem_Req), is the output-side of a FIFO to communicate instruction-fetch requests to Instruction Memory (IMem).

  The sub-interface fo_IMem_req is the corresponding return path for responses.
- The sub-interfaces fo_DMem_req and fi_DMem_rsp are the corresponding paths to/from Data memory (for LOAD/STORE instructions).

- ... ignore the remaining sub-interfaces for now; *e.g.*, the sub-interfaces fo_DMem_S_req and fi_DMem_S_rsp are paths to/from Data memory for "speculative" LOADs and STOREs, and are only relevant for **Fife**.

Let's read some code together: **Drum** CPU state

Repository:	`https://github.com/rsnikhil/Learn_Bluespec_and_RISCV_Design`
Files:	`Code/src_Drum/CPU.bsv`
Focus on:	`module mkCPU`

- The CPU state consists primarily of the Program Counter (`rg_pc`), the register file (`gprs`) and the Control-and-Status registers (`csrs`). (Please ignore the other registers for now.)
- Next, we see the registers holding inter-step values (`rg_Fetch_to_Decode` and so on).

  This is a bit extravagant: in our sequential CPU, only one of these will be active at time, so we could optimize by merging them into a single register capable of holding the largest of the inter-stage values, at the expense of clarity.
- Next we see some instantiations of **BSV** library FIFO modules which hold memory requests and responses for Fetch ("IMem") and for LOAD/STORE memory ops ("DMem").

  This may also seem a bit extravagant: in our sequential CPU, we are going to Fetch before LOAD/STORE and vice versa, so couldn't we share memory request/response FIFOs? Yes, but we want to keep our interface identical to Fife, where the pipeline may simultanously Fetch and LOAD/STORE.
- Next we see some registers to hold information for exceptions and traps (`rg_exception`, `rg_epc`, `rg_cause`, `rg_tval`).
- The next section has registers for debugger control; please ignore this for now.

Let's read some code together: **Drum** CPU Actions

Repository:	`https://github.com/rsnikhil/Learn_Bluespec_and_RISCV_Design`
Files:	`Code/src_Drum/CPU.bsv`
Focus on:	`module mkCPU`, section "Major CPU actions"

The bulk of the module body defines the major CPU action functions that we had invoked in the CPU FSM.

- Action `a_Fetch` invokes `fn_Fetch` (in file `src_Common/Fn_Fetch.bsv`), passing it the current PC (`rg_pc`).

  The result `y` (of type `Result_F`, file `src_Common/Fn_Fetch.bsv`) contains two structs, a memory-request, and information for the Decode step. The former is placed in register `rg_Fetch_to_Decode`; the latter is enqueued on the FIFO `f_IMem_req`.

- The remaining functions are similar in flavor, each invoking some `fn_XXXX` function that we have already studied.

- Action `a_Retire_direct` handles CSRxx, MRET, ECALL, EBREAK instructions.

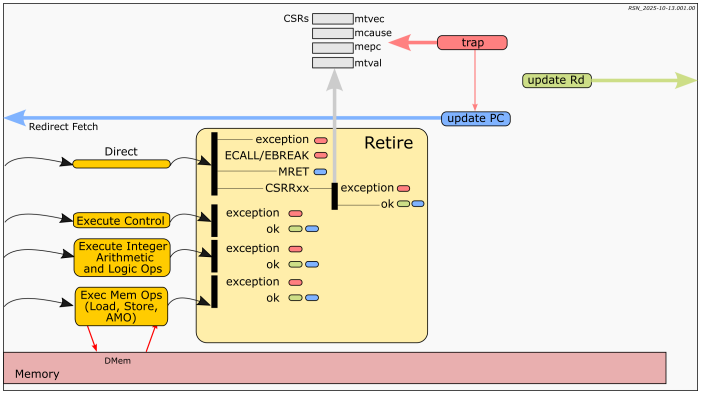
- Action `a_Retire_DMem` shows how we deal with memory exception responses, and responses with data of different sizes (Byte, Halfword, Word), with or without sign-extension.

- In the final "interface" section of the module, a line like:

  ```
  interface fo_IMem_req = to_FIFOF_O (f_IMem_req);
  ```

  simply exports the "output side" of FIFO `f_IMem_req` to the CPU interface (sub-interface of `CPU_IFC`).

Retire actions in **Drum**



- There are many alternate actions in **Drum**'s instruction-final retire actions, each taking care of some particular situation that preceded retirement.
- The rule actions share many common actions, indicated by the colored lozenges in the diagram.
- *E.g.,* If the Direct struct indicates a CSRRxx instruction, we execute the CSR function. If that raises an exception, we do the trap action, else we update Rd and update PC.
- *E.g.,* If the Direct struct indicates a LOAD/STORE instruction, and the DMem response is an exception we do the trap action, else we update Rd and update PC.

Pause ...

... and we're done reading **Drum** code!

Modulo minor syntactic differences, it looks like a RISC-V simulator program you'd write in many a popular software programming language.

This code can be compiled to Verilog which, in turn, can be built into a Verilog simulation executable, or taken through FPGA or ASIC flows into real hardware.

And that's next on the agenda ...

Contents

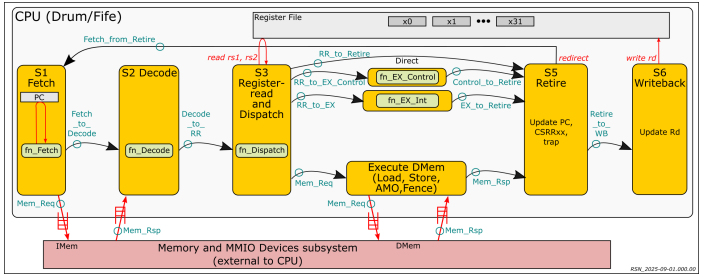
Drum: Generating Verilog, Simulation

It's important to take pause
for instant gratification (some fun),
So let's simulate often,
it's just a compile-and-run;

In BSV, like in OCaml
and in Haskell before,
If it type-checks and compiles,
it'll run, for almost sure.

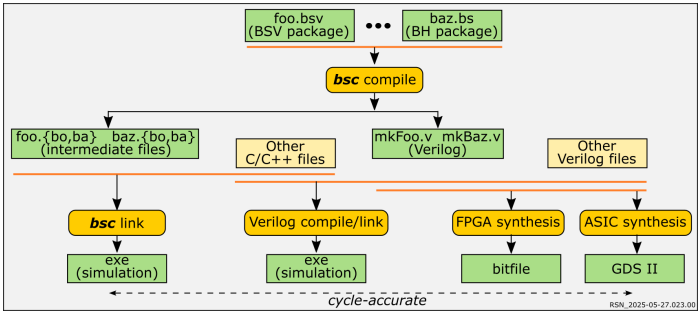
The "CPU-surround" (shared by **Drum** and **Fife**)

Repository:	`https://github.com/rsnikhil/Learn_Bluespec_and_RISCV_Design`
Directory:	`Code/src_Top/`
Files:	`Top.bsv`, `Mems_Devices.bsv`, supporting C files



- To make a simulation executable we need to connect the CPU interfaces to a CPU-surround (the purple box in the diagram).
- At a minimum, this should include a memory model
- Optional extras include a UART device (for console terminal I/O), a timer device (for measuring time and for timer interrupts), *etc.*
- This code is written in a mixture of **BSV** and C.

Build Flows for Simulation



The *bsc* compiler is typically used in one of two ways, based on command-line flags:

- *either* compile and link **BSV** into a native-code simulator (**BlueSim** simulation)
- *or* compile **BSV** into Verilog which can then be run in any Verilog simulator.

  bsc can also be used in a second step to link a Verilog simulation executable, for many well-known Verilog simulation tools (Verilator, Icarus iVerilog, CVC, Verilog simulators from Cadence, Synopsys, Siemens, *etc.*).

  The generated Verilog is standard, and can also be processed for FPGA or ASIC ("synthesis").

Let's build and run a **Drum** simulation, using Bluesim

Repository:	`https://github.com/rsnikhil/Learn_Bluespec_and_RISCV_Design`
Directory:	`Code/Build/Drum`
Using:	`Makefile` (which mostly just "include"s `../Include.mk`)

- Let's use the *bsc* compiler on the **BSV** sources:

```
$ make b_compile
bsc -u -sim    ... misc bsc flags ...
    -p     ... directory path where bsc can find source files ...
    ../../src_Top/Top.bsv
```

  ... creates various intermediate files in the `build_b/` directory.

- Let's use the *bsc* compiler to link the intermediate files:

```
$ make b_link
bsc -sim    ... misc bsc flags ...
    -e mkTop -o ./exe_Drum_RV32_bsim
      ... misc C files for the CPU-surround
```

  ... creates the executable `./exe_Drum_RV32_bsim`

- Let's run the simulation on the famous "Hello World!" program (C, compiled to RISC-V binary):

```
$ make b_run_hello
ln -s -f ../../Tools/Hello_World_Example_Code/hello.RV32.bare.memhex32 test.memhex32
./exe_Drum_RV32_bsim
... misc banner material ...
================================================================
Drum v0.81 2024-08-09 (FSM): starting execution at PC 80000000
================================================================
Hello World!
↑C
```

Let's build and run a **Drum** simulation, using Verilog (Verilator)

Repository:	`https://github.com/rsnikhil/Learn_Bluespec_and_RISCV_Design`
Directory:	`Code/Build/Drum`
Using:	`Makefile` (which mostly just "include"s `../Include.mk`)

- Let's use the *bsc* compiler on the **BSV** sources to generate Verilog:

```
$ make v_compile
bsc -u -elab -verilog    ... misc bsc flags ...
     -p    ... directory path where bsc can find source files ...
     ../../src_Top/Top.bsv
```

  ... generates Verilog files in the `verilog/` directory.
  You may wish to peruse this (later, at your convenience).

- Let's ask *bsc* to build a simulation executable (using Verilator):

```
$ make v_link
bsc -verilog -vsim verilator -use-dpi -vdir verilog    ... misc bsc flags ...
     -e mkTop -o ./exe_Drum_RV32_verilator
        ... misc C files for the CPU-surround
```

  ... creates the executable `./exe_Drum_RV32_verilator`

- Let's run the simulation on the famous "Hello World!" program (C, compiled to RISC-V binary):

```
$ make v_run_hello
ln -s -f ../../Tools/Hello_World_Example_Code/hello.RV32.bare.memhex32 test.memhex32
./exe_Drum_RV32_verilator
... misc banner material ...
================================================================
Drum v0.81 2024-08-09 (FSM): starting execution at PC 80000000
================================================================
Hello World!
↑C
```

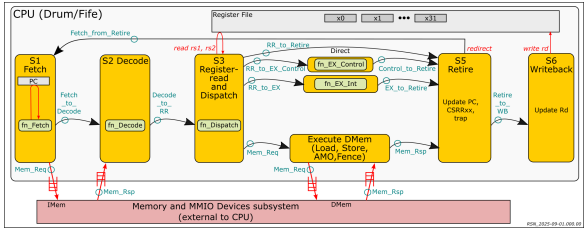
Contents

Fife *6+ stage pipeline*

And now let's pipeline it!
Naive composition would go down flaming;
Order, Prediction, Hazards, Speculation,
Each issue needs careful taming.

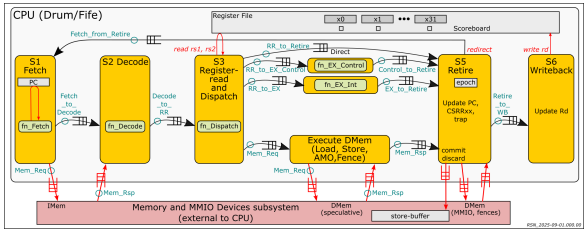
It's a bit complex; but like many things
That may initially intimidate,
Once you know how you'll say, "Is that it?",
Some of all fears will abate.

The **Fife** pipeline



In **Drum**, this figure represents the sequential states of an FSM, *i.e.*, the steps taken by an FSM, one after the other.

- One instruction is executed completely, going left to right through the figure, before we start again on the left, for the next instruction.

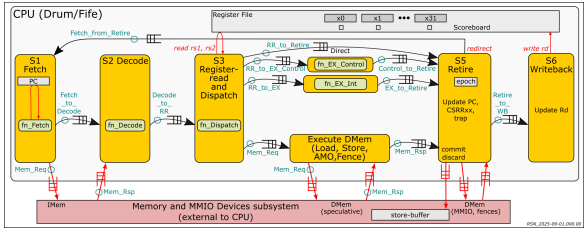


In **Fife**, this same figure represents a *pipeline*:

- Each module (Fetch, Decode, ...) is an infinite *process*, and they all run concurrently. Each is called a "pipeline stage".
- All connections between modules are now FIFOs.
- Instructions "stream" through the figure, from left to right.
- At any given point in time, there could be several instructions "in flight", in the various stages, in various states of completion.

NOTE: Fife invokes exactly the same core pure functions as **Drum**

The **Fife** pipeline: new concerns

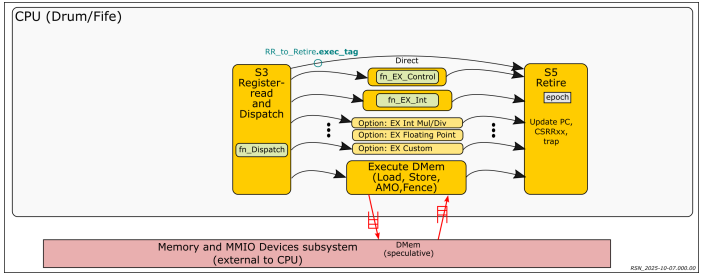


The **Fife** pipeline raises four new concerns:

- "In-order retirement": the Execute pipelines (Stage 4, or S4) may operate at different speeds (number of clock cycles), so the order in which their results arrive at S5 may be different from the order in which they entered S4 from S3. But instructions must (logically) be performed in order.
- PC prediction to keep the pipeline full, and recovery from misprediction.
- Managing register "hazards": correct ordering of reads and writes of GPRs for different instructions.
- Speculative memory STOREs: `EX_DMem` may execute a LOAD/STORE instruction I_n before we have completed instruction I_{n-k} that is logically ahead of it (in one of the other S4 pipelines). If I_{n-k} is discarded (misprediction, trap, interrupt), we should discard I_n, *i.e.,* its STORE side-effect should not be performed.

We describe the solutions in the next few slides.

The **Fife** pipeline: In-order retirement



The Execute pipelines (Stage 4, or S4) may operate at different speeds (number of clock cycles), so the order in which their results arrive at S5 may be different from the order in which they entered S4 from S3. But instructions must (logically) be performed in order.

Note that each of the S4 paths is FIFO-like (in order) even though their transit latencies may vary.

Solution:

- In Stage 3 (S3), for *every* instruction, we communicate a struct on the "Direct" path.

  We optionally *also* communicate a struct on one of the other paths (INT, Control, DMem) if it is that kind of instruction.
- In Stage 5 (S5), we use the struct at the head of the Direct path to decide whether or not to consume a struct from one of the other paths (INT, Control, DMem), and we retire the instruction.

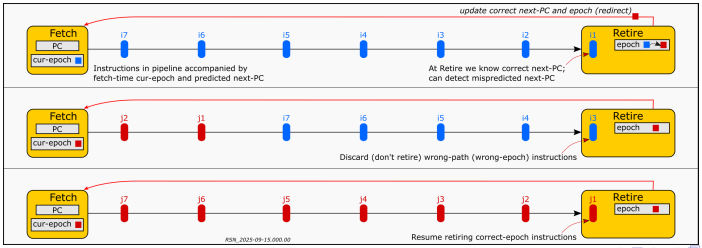
The **Fife** pipeline: PC prediction and recovery from misprediction

In Stage 1 (S1) ("Fetch"), when we issue a memory request for an instruction, the *only* information we have about it is the PC.

What PC should we use to fetch the next instruction, in order to keep the pipeline full? Normally the next PC is PC+4, but not if the fetched instruction turns out to be a (taken) BRANCH/JAL/JALR/ECALL/EBREAK, or if the instruction traps (memory fault on Fetch or LOAD/STORE, illegal instruction, misaligned BRANCH/JAL/JALR target), or if we take an interrupt, *etc.* These outcomes are only known in later stages.

When we finally know that an instruction's successor was mispredicted, how do we recover?

- In S1, we *predict* (guess) the next PC so we can do the next fetch. In **Fife** we blindly predict PC+4. *Much* better prediction is possible by analyzing the history of PCs; this is left for your future study.
- In Stage 5 (S5), we maintain an "epoch" register. When we know there was a misprediction, we increment the epoch, and send a struct to S1 indicating the correct PC and new epoch.
- S1 starts fetching from the correct PC. For each instruction, its epoch accompanies it through the pipeline.
  In S5, we discard all instructions of the wrong epoch, and start retiring again when we see the correct epoch.

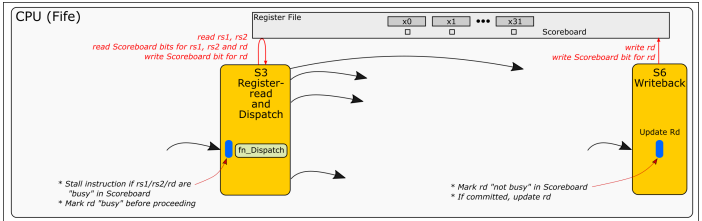


The **Fife** pipeline: Managing register "hazards"

Suppose an instruction I_n is in S4 or S5, and will write a result to GPR[x7].

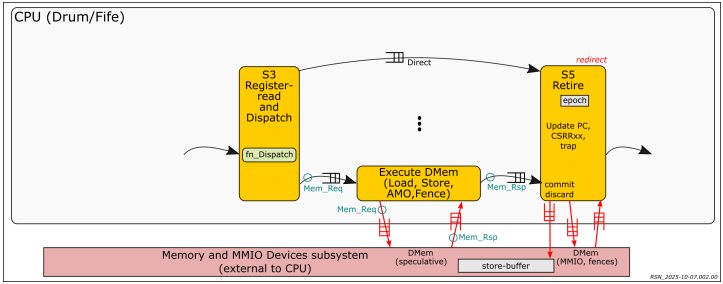
Suppose a later instruction (say, I_{n+1}) tries to read from GPR[x7]. To avoid reading a stale value, it needs to wait ("stall") until I_n has updated GPR[x7]. The danger of accessing the wrong GPR[j] value is called a "hazard".

- Along with the GPRs, we maintain, for each GPR, a 1-bit "busy" value. The collection of 32 bits is called a "scoreboard".
- When an instruction arrives at S3 we wait (stall), if necessary, until its registers are not busy. Then, we can read its Rs1 and Rs2 registers. If it has a destination register Rd, we mark it busy before allowing the instruction to move on to S4.
- In S5, when we retire an instruction, if it has an Rd, we send a struct back to S3 to free its busy bit (thereby possibly releasing an instruction might have been stalled waiting for this.



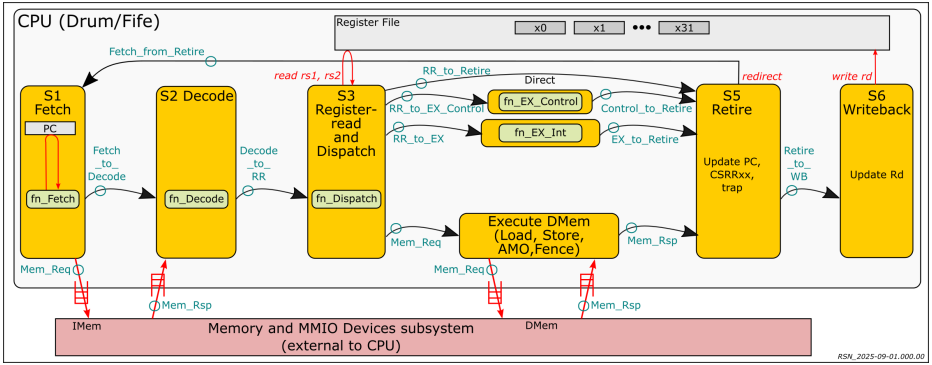
The **Fife** pipeline: Speculative memory LOADs and STOREs

`EX_DMem` may execute a LOAD/STORE instruction I_n before we have completed instruction I_{n-k} that is logically ahead of it (in one of the other S4 pipelines). If I_{n-k} is discarded (misprediction, trap, interrupt), we should discard I_n, *i.e.,* its STORE side-effect should not be performed.



- In the Memory System (outside the CPU) we maintain a "Store Buffer", where we keep an in-order record of any STORE memory updates.
- For a LOAD, we check the store buffer for the most recent update, if any, for the address of interest, and only do a usual memory access if there is no such record.
- In S5, when we retire this instruction, we send a "Commit/Discard" struct to the Store Buffer indicating how to dispose of its oldest entry: store it in memory, or discard.
- For MMIO addresses, even LOADs can have side-effects. We punt: the DMem_Speculative port does nothing but return the memory request with a DEFERRED indication, and we perform the operation in S5 once we know we're committing this instruction (using a small sequential FSM).

(Repeating earlier diagram, for convenience)

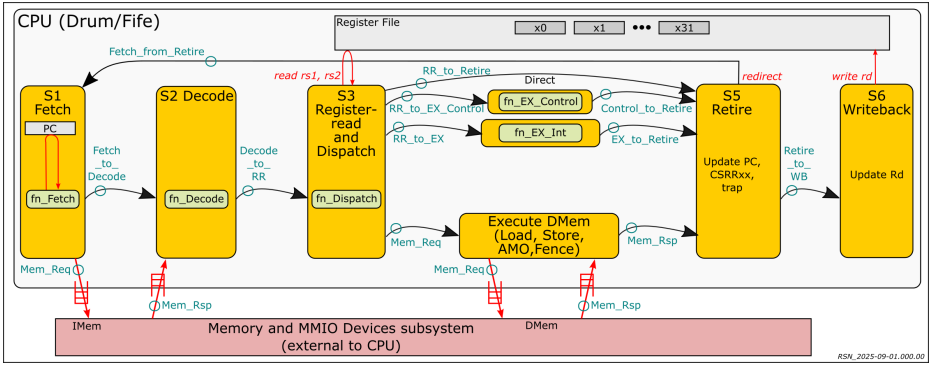


Let's read some code together: **Fife**'s S1 stage (fetch)

Repository:	`https://github.com/rsnikhil/Learn_Bluespec_and_RISCV_Design`
Files:	`Code/src_Fife/S1_Fetch.bsv`
Focus on:	`interface Fetch_IFC`, `module mkFetch`, `rl_Fetch_req`

- In interface `Fetch_IFC` there are two outgoing FIFOs (to Decode and to Memory) and one incoming FIFO (PC redirection from S5).
- In module `mkFetch` we instantite FIFOs for these interfaces.
- The rules `rl_Fetch_req` and `rl_Fetch_from_Retire` are infinite loops (processes). Each rule blocks (stalls/waits) until its rule-condition is true and all the implicit conditions of the methods it invokes are true; then it executes the rule body once; and this process repeats.
  For example, the implicit condition of a FIFO `.enq` method is true only when there is space available in the FIFO (not full). The implicit condition of IFO `.first` and `.deq` methods are true only when there is data available in the FIFO (non-empty).
- Rule `rl_Fetch_req` uses the current PC to invoke function `fn_Fetch()`, and enqueues its two results into the two outgoing FIFOs. It predictst the next PC as PC+4 (this line can be replaced to use a more sophisticated predictor that knows about the history of past PCs).
- Rule `rl_Fetch_from_Retire` handles redirections (struct that was sent from S5). It updates the PC and the epoch registers, affecting future fetches by rule `rl_Fetch_req`.

(Repeating earlier diagram, for convenience)



Let's read some code together: **Fife**'s S3 and S6 stages (register read and dispatch, and register writeback)

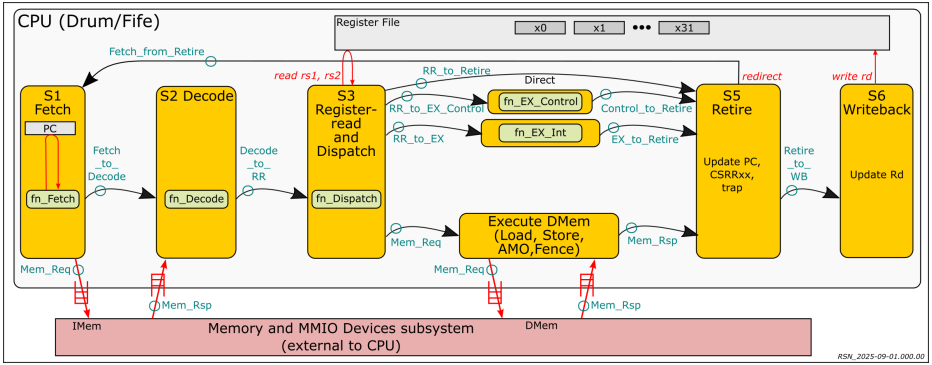
Repository:	`https://github.com/rsnikhil/Learn_Bluespec_and_RISCV_Design`
Files:	`Code/src_Fife/S3_RR_S6_WB.bsv`
Focus on:	`interface RR_WB_IFC`, `module mkRR_WB`

- In interface `RR_WB_IFC` there is one input FIFO (from Decode), one output FIFO for each of the paths through S4, and one input FIFO for register and scoreboard updates from S5.
- In module `mkFetch` we instantite FIFOs for these interfaces.
- The rule `rl_RR_Dispatch` (stage S3) checks the scoreboard if the instruction's Rs1, Rs2 or Rd (if it has such register fields) are busy.
  In the if-then-else that follows, if the scoreboard indicates we must stall, then we do nothing. This rule will try again, on the next clock
  If the scoreboard permits us to proceed, we read the rs1 and rs2 registers and invoke `fn_Dispatch()` to produce the structs to be sent to the next stage.
  Note that we blindly read GPRs rs1 and rs2, whether or not the instruction has such fields. There is no harm done if the instruction does not have such fields, since we only use them if relevant. Avoiding an unnecessary conditional will save us a MUX (multiplexor), which can be an expensive in hardware.
- Rule `rl_WB_from_Retire` (stage S6) accepts a struct sent by S5 and updates the scoreboard to "unbusy" a register (whether the S5-retired instruction was speculative or not), and updates the register if it was not speculative.

Fife's speculative LOADs and STOREs

- These are handled by the "Store_Buffer" in the memory system, which is outside the CPU.
- Being outside the CPU, and therefore not our focus today, we do not discuss this further here. In fact, we handle it in imported C code. If interested, please see the files:
  `Code/src_Top/Mems_Devices.bsv`
  `Code/src_Top/C_Mems_Devices.c`

(Repeating earlier diagram, for convenience)



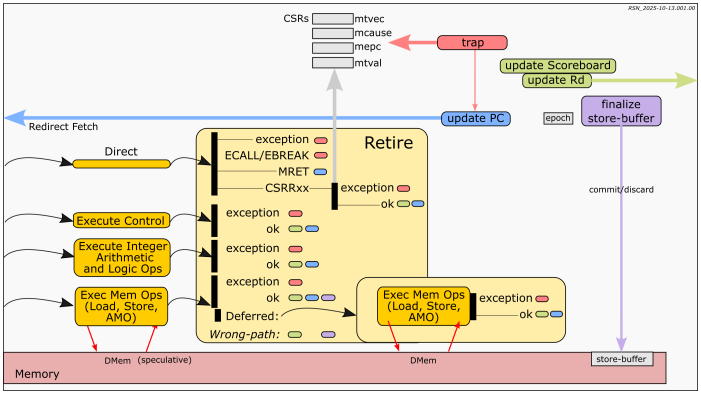
Let's read some code together: Interface for **Fife**'s S5 stage (retire/commit)

Repository:	`https://github.com/rsnikhil/Learn_Bluespec_and_RISCV_Design`
Files:	`Code/src_Fife/S5_Retire.bsv`
Focus on:	interface `Retire_IFC`

- The interface `Retire_IFC` contains:
  - An input FIFO from S3: `fi_RR_to_Retire`
  - Input FIFOs from S4: `fi_EX_Control_to_Retire`, `fi_EX_Int_to_Retire` and `fi_DMem_S_rsp` (speculative DMem response)
  - An output FIFO `fo_DMem_S_commit` (speculative DMEM commit/discard)
  - An output FIFO `fo_DMem_req` and input FIFO `fi_DMem_rsp` for MMIO (non-speculative) memory accesses
  - An output FIFO `fo_Fetch_from_Retire` to S1 for PC redirections
  - An output FIFO `fo_WB_from_Retire` to S3 for updating the GPRs and scoreboard.

- The remaining sub-interfaces are for a timer and timer interrupt, and debugger control; we do not have time to discuss these today.

Actions in module implementing **Fife**'s S5 stage



- There are *many* rules in the S5 `mkRetire` module, each taking care of some particular situation on the four input FIFOs.
- The rule actions share many common actions, indicated by the colored lozenges in the diagram.
- The "update PC" action (blue) is only performed on a misprediction.

- *E.g.,* If the Direct path carries a CSRRxx instruction, we execute the CSR function. If that raises an exception, we do the trap action, else we update Rd/Scoreboard.
- *E.g.,* If the Direct path carries a LOAD/STORE instruction, and the DMem response is DEFERRED (due to MMIO address), we now execute the memory ops. If this raises an exception, we do the trap action, else we update Rd/Scoreboard.
- *E.g.,* if the Direct path indicates a wrong-path instruction (wrong epoch), if necessary we pop one of the other three incoming FIFOs; if necessary we update the Scoreboard; if necessary we finalize the store-buffer by sending it a "Discard" message.

Let's read some code together: Module implementing **Fife**'s S5 stage

Repository:	`https://github.com/rsnikhil/Learn_Bluespec_and_RISCV_Design`
Files:	`Code/src_Fife/S5_Retire.bsv`
Focus on:	... rules (see below) ...

- Function `fa_redirect_Fetch` is a help-function invoked from numerous rules to create a redirection struct and send it to S1 (enqueue on the outgoing FIFO to S1).
- Similarly, function `fa_update_rd` is a help-function invoked from numerous rules to create a GPR/scoreboard-update struct and send it to S3 (enqueue on the outgoing FIFO to S3).
- `Bool wrong_path` is the condition of whether the next incoming instrucion from S4 is to be discarded (wrong epoch) or committed.
  Rule `rl_Retire_wrong_path` performs the action of discarding wrong-path instructions including, if necessary, releasing any busy register in the scoreboard (message to S3) and discarding any pending STORE in the Store Buffer.
- Rule `rl_Retire_Direct_exception` retires any exception that was discovered in stages S2-S3 (including fetch memory-access errors and illegal instructions).
- Rules `rl_Retire_EX_Control` and `rl_Retire_EX_Int` retire BRANCH, JAL, JALR and integer ALU instructions that came through the `EX_Control` and `EX_Int` S4 paths.
- Rule `rl_Retire_EX_DMem` commits LOAD/STORE instructions that were performed speculatively (*i.e.,* not DEFERRED).
- Rule `rl_Retire_DMem_deferred` handles DEFERRED memory instructions, now issuing the memory request. Rule `rl_Retire_DMem_rsp` handles the response.
- In any of the preceding rules, if there was an exception (trap or interrupt), they set up rule `rl_exception` to handle it.

Pause ...

... and we're done reading **Fife** code!

Congratulations!

The key difference from a traditional programming language is the execution semantics based on *concurrent rules*, a.k.a, *Guarded Atomic Actions*, as opposed to traditional sequential statements.

This code can be compiled to Verilog which, in turn, can be built into a Verilog simulation executable, or taken through FPGA or ASIC flows into real hardware.

And that's next on the agenda ...

Contents

Fife: Generating Verilog, Simulation

It's important to take pause
for instant gratification (some fun),
So let's simulate often,
it's just a compile-and-run;

In BSV, like in Haskell
and OCaml before,
If it type-checks and compiles,
it'll run, for almost sure.

While Drum works correctly,
Its execution is at best galumphing
When we pipeline it (in Fife),
We get its juices pumping.

Fife: Generating Verilog. Simulation.

Please refer back to Section 4 (page 38) for the steps to build and run a **Drum** simulation.

The steps for **Fife** simulation are exactly the same, just substitute "Fife" wherever you see "Drum" in the previous instructions.

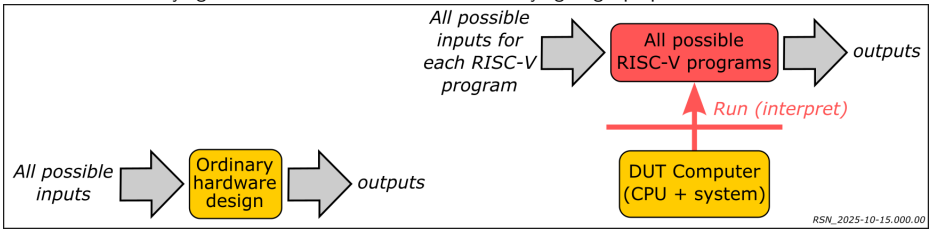
Contents

CPU Verification

And now comes the part that does
HW designers and managers terrify
We can't be sure it's right
Until our designs we verify.

Complexity of CPU verification

Verifying a CPU is often much harder than verifying single-purpose hardware IP:



... because of the extra layer of interpretation ...

Verifying an individual hardware IP may be hard because the space of

- all possible inputs

may be very large (infeasible to test all inputs).

A CPU almost always has a *very* large space of inputs:

- all possible RISC-V programs;
- for each such program, all possible inputs for that program

Traditional Verification Methodologies for CPUs

Please see the book for Chapters 11 and 12 for a discussion on traditional hardware verification methodologies for CPUs.

Some of the methodologies are just like verifying software:

- Insert "printfs" ("`$display`", "`$write` in **BSV**) at strategic points in the source code.

  See also Sections 11.3.1 and 11.3.2 for **BSV**'s `FShow` and `Fmt` facilities for automatic "pretty-printing" of enums, structs, unions, and vectors.
- Run (simulate) the design on many possible test inputs and capture the $display outputs in a log file.
- Study/analyze the log to identify erroneous behavior.
- Diagnose, fix, rebuild, and repeat.

There are sophisticated packages available from many sources to automate test generation and analysis;, a famous one is called **UVM**.

Debugging with Waveforms: **BSV** simulations (in Bluesim or Verilog simulation) can also produce so-called **VCD** files which record all bit transitions during the simulation. These **VCD** files can be viewed in any *Waveform Viewer* which displays the waveforms for each signal in the hardware.

CAUTION: In the software world, the term "verification" often refers exclusively to so-called *formal verification*, where a formal tool such as a theorem prover is used to *prove* correctness based on the semantics of the hardware design language, and not using simulation.
In the hardware world, the term "verfication" has traditionally been used for what the software world calls "testing", *i.e.*, running simulations on a suite of test inputs.

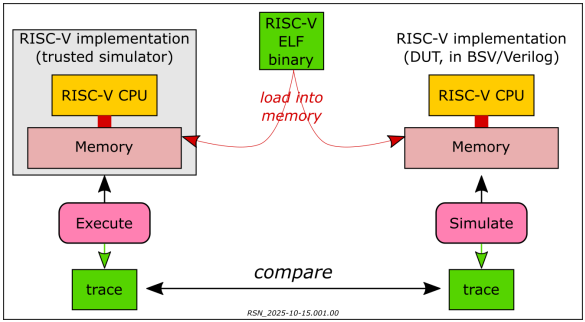
Symmetric Tandem Verification

This is a common technique for verifying CPUs.

- Relies on availability of a "trusted" simulator for the ISA.

  Example trusted simulators for the RISC-V ISA:

  - The official RISC-V ISA Formal Specification, written in the language **Sail**, which can be compiled and executed as a simulation.
  - Spike, QEMU (widely used simulators for RISC-V, written in C++)
- A single RISC-V binary is executed on both the trusted simulator and on the CPU design. Both the simulator and CPU design report the "outcomes" of each instruction (they both have to be instrumented to provide these outputs):
  - Register updates, if any (GPRs, FPRs, CSRs)
  - Memory updates, if any
  - Next PC
  - ...
- On an instruction-by-instruction basis, we compare the outcomes from the trusted simulator and the CPU design. Any discrepancy is likely a bug in the CPU design. (Comparison is automated, of course, to be able to cover millions of instructions.)



(See book Section 12.6.4 for "Asymmetric" Tandem Verification, which can also accommodate non-deterministic execution due to interrupts and MMIO reads.)

Contents

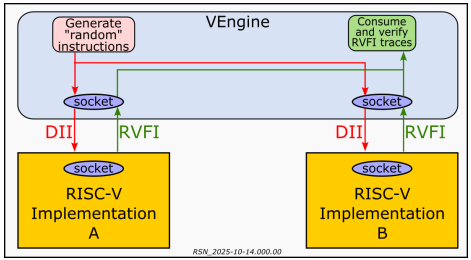
Verification using TestRIG

Fear not, fellow designers
Don't writhe in verification brig,
From our friends at U.Cambridge,
There's a lovely tool called TestRIG.

Much simpler than hand-writing
a few clever tests or formal assertions,
It will generate a zillion tests,
without requiring our exertions.

By shrinking counterexamples automatically
Debugging becomes much easier,
We reach high assurance much sooner,
Our stomachs much less queasier.

About **TestRIG**



TestRIG is a system for automated randomized testing of RISC-V implementations:

Joannou, A., Rugg, P., Woodruff, J., Fuchs, F.A., van der Maas, M., Naylor, M., Roe, M., Watson,R.N.M, Neumann, P.G. and Moore, S.W., *Randomized testing of RISC-V CPUs using direct instruction injection*, IEEE Design & Test 41:1, February 2024, pp.40-49, DOI 10.1109/MDAT.2023.3262741

TestRIG is free and open-source software, available at:
`https://github.com/CTSRD-CHERI/TestRIG`

The VEngine generates randomized sequence of instructions.

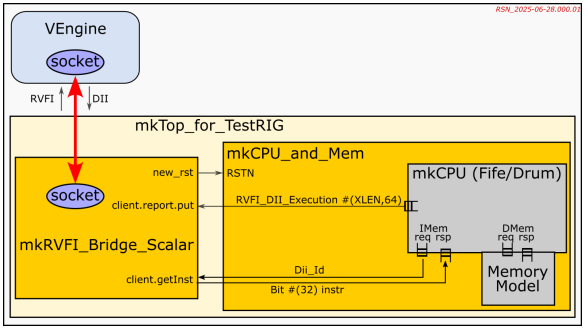
These instruction sequences are fed into two RISC-V implementations (A and B) using *Direct Instruction Injection* (DII). Typically,

- Implementation "A" is a RISC-V "reference model". Here we use a simulator built from the official *RISC-V Formal Specification* written in **Sail** (`https://github.com/riscv/sail-riscv`).
- Implementation "B" is the "Design Under Test" (or DUT). Here, it is a **Fife** or **Drum** simulation.

Both A and B report the outcomes of *each* instruction to VEngine in the **RVFI** format ("RISC-V Verification Interface"). The VEngine compares them and reports any discrepancy.

Bonus: A failing instruction sequence could be hundreds/thousands of instructions long. The VEngine will automatically try to *shrink* these sequences, often down to just a handful of instructions, making diagnosis of the error *much* easier.

Fitting **Fife** (or any RISC-V CPU design) into **TestRIG**



- `mkCPU` is instrumented to collect the information for the `RVFI_DII_Execution` struct. These structs are streamed out on a new FIFO output interface `fo_rvfi_reports` added to the `CPU_IFC` interface.
- We create a new module `mkCPU_and_Mem` (file `TestRIG/src_Top_TestRIG/CPU_and_Mem.bsv`).

  Unlike the normal `Top.bsv` for standalone simulation, here **we do not connect the IMem memory interfaces (for Fetch) to the memory model**.

  Instead, we have logic that, for every IMem request, consumes an instruction from **TestRIG** and feeds that in the IMem response (we ignore the instruction address (PC) in the request).
- `mkRVFI_Bridge_Scalar` is from U.Cambridge GitHub:
  `https://github.com/CTSRD-CHERI/BSV-RVFI-DII.git`
  We obtain a local copy (see `TestRIG/vendor/`).
- We create a new top-level BSV module `mkTop_for_TestRIG` (file `TestRIG/src_Top_TestRIG/Top_TestRIG.bsv`) that instantiates `mkRVFI_Bridge_Scalar` and `mkCPU_and_Mem`, and connects up the reset, `report` and `getInst` interfaces.

And that's it! We're ready to roll!

Let's read some code together: instrumenting the CPU for RVFI reports

Repository:	`https://github.com/rsnikhil/Learn_Bluespec_and_RISCV_Design`
Files:	`Code/vendor/RVFI_DII_Types/RVFI_DII_Types.bsv`
	`Code/src_Common/RVFI_Reports.bsv`
	`Code/src_Fife/S5_Retire.bsv`
Focus on:	... (see below) ...

- File `RVFI_DII_Types.bsv` defines the struct type `RVFI_DII_Execution` for the RVFI "Reports" sent by the CPU to VEngine for each instruction. For each kind (opcode) of instruction, only some fields will be relevant.
- Module `mkRVFI_Report` (file `Code/src_Common/RVFI_Reports.bsv`) contains "help" methods for producing the RVFI report for each kind of instruction. They're all very similar; take a look at `rvfi_Int` for example.
- Module `mkRetire` (file `Code/src_Fife/S5_Retire.bsv`) instantiates module `mkRVFI_Report`.
- In module `mkRetire` (file `Code/src_Fife/S5_Retire.bsv`), look at rule `rl_Retire_EX_Int` and observe the invocation of `rvfi_report.rvfi_Int()` to create the RVFI report for integer (ALU) instructions.

Let's read some code together: mkCPU_and_Mem

Repository:	https://github.com/rsnikhil/Learn_Bluespec_and_RISCV_Design
Files:	TestRIG/src_Top_TestRIG/CPU_and_Mem.bsv
Focus on:	... (see below) ...

- Interface CPU_and_Mem_IFC has the three sub-interfaces shown in the diagram on an earlier slide.
- Module mkCPU_and_Mem instantiates:
  - mkCPU (modified only with instrumentation to produce RVFI_DII_Execution reports for each instruction).
  - mkMems_Devices (unmodified). Note that we pass "dummy" FIFO interfaces for IMem_reqs and IMem_rsps, where previously we would have passed in the corresponding sub-interfaces from mkCPU.
- Module mkCPU_and_Mem's rule rl_step1, which is executed during startup, invokes mems_devices.init(init_params), where we have supplied the memory base address and memory size expected by **TestRIG**.
- Module mkCPU_and_Mem's rule rl_relay_instrs gets a memory Fetch request from the CPU, consumes an instruction from VEngine (the f_instrs FIFO), and sends a memory Fetch response to the CPU.
- Module mkCPU_and_Mem's rule rl_relay_rvfi relays the RVFI_DII_Execution reports from the CPU out to mkRVFI_Bridge_Scalar
- **TODO: explain how we deal with "wrong-path" Fetch requests.**

Let's read some code together: mkTop_for_TestRIG

Repository:	https://github.com/rsnikhil/Learn_Bluespec_and_RISCV_Design
Files:	TestRIG/src_Top_TestRIG/Top_TestRIG.bsv
Focus on:	... (see below) ...

The module mkTop_TestRIG is the top-level (Empty interface) of the simulation executable for running with **TestRIG**.

- It instantiates modules mkRVFI_DII_Bridge_Scalar and mkCPU_and_Mem. In the latter, "reset_by bridge.new_rst" phrase allows it to be reset by the former (this is invoked after each test run by **TestRIG** so that each test begins with the CPU in a known, predictable state).
- The two rules merely forward the DII instruction traffic from VEngine to CPU, and the RVFI_DII_Execution reports from the CPU to the VEngine.

Demo: **TestRIG** comparing **Sail** RISC-V Formal Spec and **Fife**

Start the DUT simulation (**Fife** with its **TestRIG** wrapper) in one window.

Start **TestRIG** in another window.

- Study and discuss command-line arguments for **TestRIG**
- Note that the "A" comparand is automatically started by **TestRIG**, and is an executable created from the **Sail** RISC-V Formal Specification.
  The "B" comparand (**Fife**) could also be automatically started by **TestRIG**, but we have started it manually here just for demo purposes.
- Study and discuss **TestRIG**'s output.
- Deliberately introduce a bug in **Fife**, and repeat a run; study and discuss **TestRIG**'s "shrinking" and discrepancy-reporting.

Comparing **TestRIG** with other verification methodologies

Compare: Random program generation (`https://github.com/chipsalliance/riscv-dv`)

- Easier random generation by bypassing the instruction-fetch mechanism (no need to generate "control-flow-correct" random programs).
- Shrinkage of tests to "minimal" counter-examples.
- Very powerful generator mechanisms (Haskell QuickCheck library).

Compare: ad-hoc test programs:

- *Much* larger repertoire of tests.
- *Many* more corner cases explored.
- Easier random generation by bypassing the instruction-fetch mechanism (no need to generate "control-flow-correct" random programs).
- Shrinkage of tests to "minimal" counter-examples.

Compare: Formal Verification tools (*e.g.*, Jasper):

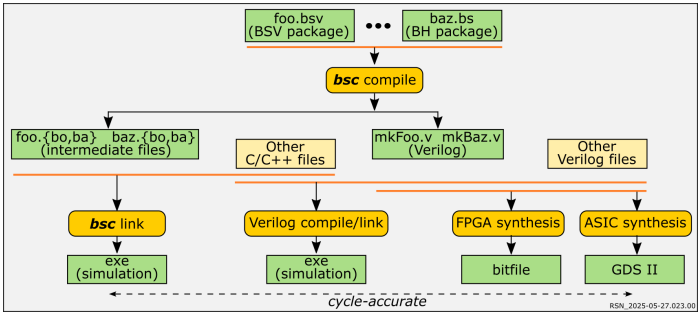
- **TestRIG** is automatically checkeing a gazillion correctness properties: each test is checking the property that Fife has the same behavior as **Sail**. With Formal Verification tools, users have to write all properties by hand.
- *Many* more corner cases are likely explored.

Contents

Build flows for FPGA and ASIC

Our Verilog we can take to Silicon
In ASIC or FPGA
But that's a whole subject in itself
Here we won't have much to say.

Build Flows for FPGA and ASIC



- For both FPGAs and ASICs, we proceed with the same Verilog that we have already generated for simulation, taking it through "synthesis" tools.
- Certain constructs in the Verilog are ignored by synthesis tools, notably `$display`, `$finish`, and imported C (and the synthesis tools will optimize away any logic that was only producing data for such statements).
- In both cases, we need to add additional circuitry (and possibly intervening IPs) that connect our design to the "pins" of the FPGA/ASIC, for data I/O.
- For FPGAs the output is a "bitfile" that is then loaded into the FPGA to configure the FPGA to emulate our design.
- For ASICs the output is a "GDS II" file that is sent to a fabrication facility/foundry for rendering into silicon.

Thank you!

1. Introduction
2. Core pure functions (shared by **Drum** and **Fife**)
3. **Drum**
4. **Drum**: Generating Verilog, Simulation
5. **Fife** 6+ stage pipeline
6. **Fife**: Generating Verilog, Simulation
7. CPU Verification
8. Verification using **TestRIG**
9. Build flows for FPGA and ASIC

Q & A?

Thank you!

Dear friends, about CPU design
We've just had a lightning preview,
We've left you with all the materials
As programmers you can see, its not anything new!
We hope you're inspired to go try it later
And take flight on your own! Whoo hoo!

It's been a pleasure for me to share this,
So from me, a sincere Thank You!

Links to additional resources (all free and open-source):

bsc compiler for BSV and BH, libraries, discussion forums:

https://github.com/B-Lang-org/bsc
https://groups.io/g/b-lang-discuss
https://groups.io/g/b-lang-announce/topics
https://github.com/B-Lang-org/bsc-contrib (incl. AMBA/AXI fabrics)
https://github.com/BSVLang/Main

Textbook, lecture slides, exercises, full code for **Drum** and **Fife** RISC-V CPUs (buildable, runnable):

https://github.com/rsnikhil/Learn_Bluespec_and_RISCV_Design

RISC-V CPUs (all RV64GC, Linux-capable, coded in BSV):

Piccolo (3-stage)	https://github.com/bluespec/Piccolo	
Flute (5-stage)	https://github.com/bluespec/Flute	
Toooba (Superscalar, out-of-order)	https://github.com/bluespec/Toooba	
CHERI (RISC-V + security capabilities):	https://github.com/CTSRD-CHERI	(U.Cambridge)

RISC-V IP (coded in BSV):

CLINT (timer and sw intertupts)	https://github.com/bluespec/CLINT
PLIC (interrupt controller)	https://github.com/bluespec/PLIC
Debug Module	https://github.com/bluespec/RISCV_Debug_Module
gdbstub	https://github.com/bluespec/RISCV_gdbstub

Slides preparation tools:

- Text and layout: LaTeX with Beamer package, Madrid theme
- Diagrams: Inkscape for SVG; Inkscapet to export to PNG